

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

✓ Q1

Single Layer Perceptron and Multi Layer Perceptron A Single Layer Perceptron is the simplest type of Artificial Neural Network (ANN). It contains:

- One input layer
- One output layer
- No hidden layer

It is mainly used for linearly separable problems. Input $\rightarrow$ Output Output Equation: $\hat{y} = f(\sum w_i x_i + b)$ Where:

- f = activation function
- w_i = weights
- x_i = inputs
- b = bias

For Classification: $L = -\sum y_i \log(\hat{y}_i)$ For Regression: $L = (1/n) \sum (y_i - \hat{y}_i)^2$

A Multi Layer Perceptron contains:

- Input layer
- One or more hidden layers
- Output layer

MLP can solve non-linear problems using activation functions like:

- ReLU
- Sigmoid
- Softmax

Input Layer $\rightarrow$ Hidden Layer(s) $\rightarrow$ Output Layer

$a(1) = f(W(1)x + b(1))$ $\hat{y} = f(W(2)a(1) + b(2))$

- Learns complex patterns
- Handles non-linear data
- Used in image recognition, NLP, speech recognition, etc.

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q2
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

scaler = StandardScaler()
X = scaler.fit_transform(X)
y = to_categorical(y, num_classes=3)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = Sequential()
model.add(Input(shape=(4,)))
model.add(Dense(10, activation='relu'))
model.add(Dense(3, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=10,
    verbose=1
)

y_pred = model.predict(X_test)
y_pred_labels = np.argmax(y_pred, axis=1)
y_test_labels = np.argmax(y_test, axis=1)

accuracy = accuracy_score(
    y_test_labels,
    y_pred_labels
)
print(f"Accuracy: {accuracy*100:.2f}%")
```

```

Epoch 79/100
12/12  0s 7ms/step - accuracy: 0.9333 - loss: 0.1954
Epoch 80/100
12/12  0s 8ms/step - accuracy: 0.9333 - loss: 0.1932
Epoch 81/100
12/12  0s 9ms/step - accuracy: 0.9333 - loss: 0.1908
Epoch 82/100
12/12  0s 10ms/step - accuracy: 0.9333 - loss: 0.1888
Epoch 83/100
12/12  0s 13ms/step - accuracy: 0.9417 - loss: 0.1869
Epoch 84/100
12/12  0s 13ms/step - accuracy: 0.9500 - loss: 0.1851
Epoch 85/100
12/12  0s 17ms/step - accuracy: 0.9500 - loss: 0.1825
Epoch 86/100
12/12  0s 18ms/step - accuracy: 0.9500 - loss: 0.1804
Epoch 87/100
12/12  0s 15ms/step - accuracy: 0.9500 - loss: 0.1784
Epoch 88/100
12/12  0s 7ms/step - accuracy: 0.9500 - loss: 0.1763
Epoch 89/100
12/12  0s 11ms/step - accuracy: 0.9500 - loss: 0.1746
Epoch 90/100
12/12  0s 15ms/step - accuracy: 0.9500 - loss: 0.1733
Epoch 91/100
12/12  0s 11ms/step - accuracy: 0.9500 - loss: 0.1709
Epoch 92/100
12/12  0s 6ms/step - accuracy: 0.9500 - loss: 0.1694
Epoch 93/100
12/12  0s 17ms/step - accuracy: 0.9500 - loss: 0.1671
Epoch 94/100
12/12  0s 17ms/step - accuracy: 0.9500 - loss: 0.1654
Epoch 95/100
12/12  0s 12ms/step - accuracy: 0.9500 - loss: 0.1641
Epoch 96/100
12/12  0s 13ms/step - accuracy: 0.9500 - loss: 0.1620
Epoch 97/100
12/12  0s 13ms/step - accuracy: 0.9500 - loss: 0.1605
Epoch 98/100
12/12  0s 14ms/step - accuracy: 0.9500 - loss: 0.1591
Epoch 99/100
12/12  0s 8ms/step - accuracy: 0.9500 - loss: 0.1573
Epoch 100/100
12/12  0s 11ms/step - accuracy: 0.9500 - loss: 0.1556
1/1  0s 108ms/step
Accuracy: 100.00%

```

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

Q3

```

import numpy as np
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import accuracy_score

```

```

(X_train, y_train), (X_test, y_test) = mnist.load_data()

```

```

X_train = X_train / 255.0
X_test = X_test / 255.0

X_train = X_train.reshape(-1, 28*28)
X_test = X_test.reshape(-1, 28*28)

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential()
model.add(Input(shape=(28*28,)))
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    verbose=1
)

y_pred = model.predict(X_test)
y_pred_labels = np.argmax(y_pred, axis=1)
y_test_labels = np.argmax(y_test, axis=1)

accuracy = accuracy_score(
    y_test_labels,
    y_pred_labels
)

print(f"Accuracy: {accuracy*100:.2f}%")

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-data/11490434/11490434>  2s 0us/step

Epoch 1/10

1875/1875  **10s** 5ms/step - accuracy: 0.9261 - loss: 0.2591

Epoch 2/10

1875/1875  **10s** 5ms/step - accuracy: 0.9662 - loss: 0.1135

Epoch 3/10

1875/1875  **9s** 5ms/step - accuracy: 0.9765 - loss: 0.0769

Epoch 4/10

1875/1875  **9s** 5ms/step - accuracy: 0.9823 - loss: 0.0577

Epoch 5/10

1875/1875  **9s** 5ms/step - accuracy: 0.9858 - loss: 0.0452

Epoch 6/10

1875/1875  **9s** 5ms/step - accuracy: 0.9893 - loss: 0.0346

Epoch 7/10

1875/1875  **8s** 4ms/step - accuracy: 0.9911 - loss: 0.0280

```
Epoch 8/10
1875/1875 ————— 10s 5ms/step - accuracy: 0.9934 - loss: 0.0222
Epoch 9/10
1875/1875 ————— 10s 5ms/step - accuracy: 0.9944 - loss: 0.0187
Epoch 10/10
1875/1875 ————— 9s 4ms/step - accuracy: 0.9952 - loss: 0.0154
313/313 ————— 1s 3ms/step
Accuracy: 97.71%
```

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

✓ Q4

Convolution, ReLU and Max Pooling

Given Input Image

[[1, 2, 0, 1], [3, 1, 2, 2], [1, 0, 1, 3], [2, 1, 2, 1]]

Kernel

[[1, 0], [0, -1]]

Step 1: Convolution

Formula: $\text{Output} = \Sigma(\text{Input} \times \text{Kernel})$

Convolution Output

[[0, 0, -2], [3, 0, -1], [0, -2, 0]]

Step 2: ReLU Activation

Formula: $\text{ReLU}(x) = \max(0, x)$

ReLU Output

[[0, 0, 0], [3, 0, 0], [0, 0, 0]]

Max Pooling Output

[[3]]

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

```
# Q5
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
```

```

        Conv2D,
        MaxPooling2D,
        Flatten,
        Dense
    )
from tensorflow.keras.utils import to_categorical

(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train / 255.0
X_test = X_test / 255.0

X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential()
model.add(
    Conv2D(
        32,
        (3,3),
        activation='relu',
        input_shape=(28,28,1)
    )
)
model.add(MaxPooling2D((2,2)))
model.add(
    Conv2D(
        64,
        (3,3),
        activation='relu'
    )
)

model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=32,
    verbose=1
)
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy*100:.2f}%")

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_c
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5

```

1875/1875  **77s** 40ms/step - accuracy: 0.9668 - loss: 0.108
Epoch 2/5
1875/1875  **74s** 39ms/step - accuracy: 0.9884 - loss: 0.037
Epoch 3/5
1875/1875  **73s** 39ms/step - accuracy: 0.9925 - loss: 0.023
Epoch 4/5
1875/1875  **83s** 39ms/step - accuracy: 0.9951 - loss: 0.015
Epoch 5/5
1875/1875  **73s** 39ms/step - accuracy: 0.9964 - loss: 0.011
313/313  **4s** 13ms/step - accuracy: 0.9906 - loss: 0.0377
Test Accuracy: 99.06%